Photography Studio Booking System

Requirements Notes — Person 8: Payment & Review

1. Scope

This document describes the requirements for the **Payment** and **Review** entities assigned to Person 8. The entities connect to the existing **Booking** and **Customer** entities, respectively. The Booking and Customer entities are referenced here as existing entities and are not redefined in this document.

2. Entity: Payment

Attribute	Requirement / Meaning
payment_id (Primary Key)	Uniquely identifies each payment. Implemented as an auto-generated integer using PostgreSQL `SERIAL`.
booking_id (Foreign Key)	References `booking(booking_id)`. Required and unique so each Payment belongs to one Booking and each Booking can have at most one Payment.
amount	Required monetary amount, stored as `NUMERIC(10,2)`. Must be greater than zero.
method	Required payment method. Allowed values: `cash`, `card`, or `online`.
payment_date	Required date on which the payment is recorded, stored as `DATE`.

3. Entity: Review

Attribute	Requirement / Meaning
review_id (Primary Key)	Uniquely identifies each review. Implemented as an auto-generated integer using PostgreSQL `SERIAL`.
customer_id (Foreign Key)	References `customer(customer_id)` and is required. Each Review belongs to one Customer.
rating	Required integer rating from 1 through 5, inclusive.
comment	Optional written feedback, stored as PostgreSQL `TEXT`.
review_date	Required date on which the review is recorded, stored as `DATE`.

4. Relationships and Cardinality

Relationship	Cardinality	Implementation
Booking — Payment	One-to-one (1:1)	The `payment.booking_id` foreign key is `NOT NULL` and `UNIQUE`. This allows at most one payment record per booking.
Customer — Review	One-to-many (1:M)	The `review.customer_id` foreign key is `NOT NULL`. A customer may have multiple reviews; each review refers to one customer.

5. Integrity Constraints and Deletion Rules

• **Primary keys:** `payment_id` and `review_id` uniquely identify rows in their respective tables.

• **Payment amount:** `CHECK (amount > 0)` prevents zero or negative amounts.

• **Payment method:** `CHECK (method IN ('cash', 'card', 'online'))` restricts the method to the agreed values.

• **Review rating:** `CHECK (rating BETWEEN 1 AND 5)` restricts ratings to integers from 1 to 5.

• **Foreign keys:** each payment must reference an existing booking, and each review must reference an existing customer.

• **ON DELETE CASCADE:** deleting a referenced Booking deletes its associated Payment; deleting a referenced Customer deletes that customer's Review rows.

6. Implementation Notes

The DDL is intended for **PostgreSQL**. The SQL file should contain table-definition statements (`CREATE TABLE` and, if needed by the team schema, `ALTER TABLE`) only, with no sample data insertion or data-manipulation statements. The referenced `booking` and `customer` tables must exist before these foreign keys can be created.

> **Cardinality clarification:** the Payment table's `UNIQUE NOT NULL` foreign key ensures that a Payment has exactly one Booking and that a Booking has at most one Payment. By itself, it does not require every Booking to have a Payment.

Prepared for Phase 1 submission • Person 8 • Payment & Review